

Aula 06 — Árvores de Regressão e *Ensembles*

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §4.8–4.10; ISLP cap. 8

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- descrever como uma **árvore de regressão** particiona o espaço de covariáveis e prediz por médias locais;
- explicar a construção em duas etapas: *crescimento* por divisões binárias recursivas e *poda* por custo-complexidade;
- justificar, via redução de variância, por que **agregar** estimadores ajuda;
- descrever **bagging**, a estimativa *out-of-bag* e a medida de importância de variáveis;
- explicar como as **florestas aleatórias** *descorrelacionam* as árvores e o papel de m ;
- descrever **boosting** como ajuste incremental dos resíduos e o papel de λ , B e da profundidade.

Em sala

Árvores são o método mais *interpretável* que veremos — e, sozinhas, das piores em predição. A história da aula é como transformá-las, via agregação, em alguns dos métodos *mais poderosos* (florestas, XGBoost). Pergunta de abertura: “*como combinar muitos modelos ruins para obter um ótimo?*”

1 Árvores de regressão

Uma árvore parte o espaço de covariáveis em regiões disjuntas $R_1, \dots, R_J$ por *particionamento recursivo* e prediz, em cada região, a média das respostas de treino ali contidas:

$$g(\mathbf{x}) = \frac{1}{|\{i : \mathbf{X}_i \in R_k\}|} \sum_{i: \mathbf{X}_i \in R_k} Y_i, \quad \text{para } \mathbf{x} \in R_k. \quad (1)$$

Para prever, percorre-se a árvore do topo às folhas seguindo as condições (“ $x_j < t$?”) em cada nó.

Ideia central

Vantagens estruturais das árvores: (i) altamente **interpretáveis**; (ii) lidam *nativamente* com covariáveis discretas (cortes do tipo $x_j \in \{\text{verde}, \text{vermelho}\}$ — sem precisar criar *dummies*); (iii) fazem **seleção de variáveis** automática; (iv) capturam **interações** sem que precisemos especificá-las.

1.1 Etapa 1 — crescimento (divisões binárias recursivas)

Idealmente escolheríamos a partição que minimiza o EQM $P(T) = \sum_R \sum_{i: X_i \in R} (Y_i - \hat{y}_R)^2 / n$. Mas buscar a árvore ótima é computacionalmente inviável (NP-difícil). Usa-se então uma heurística *gananciosa*: a cada passo, escolhe-se a covariável x_j e o corte t que mais reduzem a soma de quadrados ao dividir uma região em $R_1 = \{x_j < t\}$ e $R_2 = \{x_j \geq t\}$:

$$\min_{j, t} \sum_{i: X_i \in R_1} (Y_i - \hat{y}_{R_1})^2 + \sum_{i: X_i \in R_2} (Y_i - \hat{y}_{R_2})^2. \quad (2)$$

Repete-se recursivamente em cada região-filha, até um critério de parada (ex.: menos de 5 observações por folha). Isso gera uma árvore grande T_0 , que ajusta bem o treino mas tende a **superajustar**.

Em sala

Pergunta: “a árvore gananciosa acha a solução de $y = \text{sin}(x_1 x_2 x_3)$?” Oito regiões resolvem exatamente, e caberiam em profundidade 3 — então a resposta intuitiva é sim. A §3 da Aula prática 06 mede: o primeiro corte reduz 0,194% do RSS e o segundo, 0,350%; o R^2 de teste é **negativo** até a profundidade 5 e só chega a 0,87 com a árvore cheia, de 59 folhas. Vale desenhar os octantes no quadro e mostrar que qualquer corte deixa os dois lados com média zero.

E o corolário que interessa para a próxima subseção: com `min_impurity_decrease=0,005` a árvore fica com **uma folha**. A parada precoce não é uma poda mais barata — é uma poda que decide antes de ter informação.

1.2 Etapa 2 — poda por custo-complexidade

Para reduzir a variância, *podamos* T_0 . A poda por custo-complexidade introduz um parâmetro $\alpha \geq 0$ e busca a subárvore $T \subseteq T_0$ que minimiza

$$\sum_{k=1}^{|T|} \sum_{i: X_i \in R_k} (Y_i - \hat{y}_{R_k})^2 + \alpha |T|, \quad (3)$$

onde $|T|$ é o número de folhas. O termo $\alpha |T|$ penaliza árvores grandes — é o mesmo espírito da regularização da Aula 02 (trocamos viés por menos variância). α é o **tuning parameter**, escolhido por **validação cruzada** (Aula 03): $\alpha = 0$ devolve T_0 ; $\alpha \rightarrow \infty$ colapsa para a raiz (média global).

Atenção

Crescer e podar usam objetivos diferentes *de propósito*: cresce-se de forma gananciosa (olhando só um passo à frente) e corrige-se globalmente na poda. Não confunda profundidade máxima (parâmetro de crescimento) com α (parâmetro de poda) — ambos controlam complexidade, mas em etapas distintas.

2 Por que agregar? Redução de variância

Árvores isoladas têm *alta variância*: pequenas mudanças nos dados mudam bastante a árvore. A ideia dos *ensembles* é **combinar** muitas funções para reduzir essa variância.

Proposição 2.1 (Média de preditores reduz o risco). *Sejam $g_1, \dots, g_B$ estimadores não-viesados ($\mathbb{E}[g_b(\mathbf{x})] = r(\mathbf{x})$), de mesma variância $v(\mathbf{x}) = \text{Var}(g_b(\mathbf{x}))$ e não-correlacionados. Então a média $\bar{g} = \frac{1}{B} \sum_b g_b$ tem risco*

$$\mathbb{E}[(Y - \bar{g}(\mathbf{x}))^2 | \mathbf{x}] = \text{Var}(Y | \mathbf{x}) + \frac{v(\mathbf{x})}{B} \leq \mathbb{E}[(Y - g_b(\mathbf{x}))^2 | \mathbf{x}].$$

Demonstração. Pela decomposição viés-variância (Aula 01), o risco de qualquer preditor g é $\text{Var}(Y | \mathbf{x}) + \text{viés}^2 + \text{Var}(g(\mathbf{x}))$. Como cada g_b é não-viesado, $\bar{g}$ também é (viés = 0). Para a variância, sendo os g_b não-correlacionados e de variância $v(\mathbf{x})$, $\text{Var}(\bar{g}(\mathbf{x})) = \frac{1}{B^2} \sum_b \text{Var}(g_b(\mathbf{x})) = v(\mathbf{x})/B$. Substituindo obtém-se a igualdade; como $v(\mathbf{x})/B \leq v(\mathbf{x})$, o risco não aumenta. $\square$

Ideia central

Duas exigências da Prop. 2.1 guiam tudo o que segue: precisamos de preditores (i) **não-viesados** — por isso usamos árvores *não podadas* (profundas, baixo viés) — e (ii) **pouco correlacionados** — o ponto fraco, que as florestas aleatórias atacam diretamente. Quanto menor a correlação, mais a variância cai.

3 Bagging

Bagging (*bootstrap aggregating*) gera diversidade reamostrando os dados. Para $b = 1, \dots, B$: sorteie uma amostra **bootstrap** (com reposição, tamanho n) e ajuste nela uma árvore *profunda* (não podada) g_b . A predição é a média:

$$g(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}). \quad (4)$$

3.1 Estimativa *out-of-bag* (OOB)

Cada amostra bootstrap omite, em média, $\approx 37\%$ das observações (as *out-of-bag*, Aula 03). Podemos prever cada $\mathbf{X}_i$ usando *só* as árvores que não a viram: isso dá uma estimativa de risco **de graça**, sem CV separada.

3.2 Importância de variáveis

Bagging sacrifica interpretabilidade, mas recupera parte dela com uma medida de **importância**: para cada variável, soma-se a *redução de RSS* em todas as divisões que a usaram, $\text{RSS}_{\text{pai}} - \text{RSS}_{\text{esq}} - \text{RSS}_{\text{dir}}$, e faz-se a média sobre as B árvores. Variáveis que reduzem muito o erro são as mais importantes.

4 Florestas aleatórias

O problema do *bagging*: as árvores ficam **correlacionadas**. Se há uma covariável muito forte, quase todas as árvores a usam no topo e ficam parecidas — e a Prop. 2.1 rende pouco quando $\text{Cor}(g_b)$ é alta.

Ideia central

Florestas aleatórias (Breiman) *descorrelacionam* as árvores: em *cada nó*, sorteia-se um subconjunto de apenas $m < p$ covariáveis e a divisão só pode usar uma delas. Isso impede que a variável dominante apareça sempre, diversificando as árvores e reduzindo mais a

variância.

Detalhes:

- m é o *tuning parameter*; regra empírica $m \approx p/3$ em regressão (e $m \approx \sqrt{p}$ em classificação). $m = p$ recupera o *bagging*.
- O risco é **robusto a B** : ele estabiliza e *não há superajuste* ao aumentar B — não vale a pena fazer CV em B , basta usar B “grande o bastante”.
- Conexão com a Aula 05: a taxa de convergência das florestas depende essencialmente do número de *variáveis relevantes* — elas se adaptam à *esparsidade*.

5 Boosting

Boosting também agrega árvores, mas de forma **sequencial e adaptativa**: cada nova árvore corrige os *resíduos* da combinação atual. Não reamostra; reajusta ao erro corrente.

Algoritmo (boosting de árvores para regressão)

1. Inicialize $g(\mathbf{x}) \equiv 0$ e resíduos $r_i \leftarrow Y_i, \forall i$.
2. Para $b = 1, \dots, B$:
 - (a) ajuste uma árvore *rasa* (com p folhas) a $(\mathbf{X}_1, r_1), \dots, (\mathbf{X}_n, r_n)$; chame-a g_b ;
 - (b) atualize $g(\mathbf{x}) \leftarrow g(\mathbf{x}) + \lambda g_b(\mathbf{x})$ e $r_i \leftarrow Y_i - g(\mathbf{X}_i)$.
3. Retorne $g(\mathbf{x}) = \lambda \sum_{b=1}^B g_b(\mathbf{x})$.

Começamos com alto viés (zero) e *baixíssima* variância; cada passo *reduz o viés* adicionando um “aprendiz fraco”. Os *tuning parameters*:

λ (*learning rate*)

fator pequeno (ex.: 0,01) que controla o quanto cada árvore contribui. λ grande $\Rightarrow$ superajuste rápido; pequeno $\Rightarrow$ aprendizado lento (precisa de B maior).

p (*profundidade/folhas*)

árvores rasas ($p = 2$ a 4 ; *stumps* têm uma divisão) controlam a ordem de interações capturadas.

B (*número de árvores*)

escolhido por *early stopping*: pare quando o risco em validação parar de cair.

Atenção

Diferença crucial em relação às florestas: no *boosting*, B **grande demais superajusta** (o modelo continua reduzindo viés até ajustar ruído). Por isso o *early stopping* é essencial. Nas florestas, ao contrário, B grande é inofensivo.

Observação 5.1. A implementação moderna mais popular é o **XGBoost** (Chen & Guestrin, 2016): *boosting* de árvores com regularização adicional, tratamento de dados faltantes e altíssimo desempenho computacional — frequentemente o melhor método “pronto” em dados tabulares.

6 Prática em Python

```
1 from sklearn.tree import DecisionTreeRegressor
2 from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
3 from sklearn.model_selection import GridSearchCV
```

```
4
5 # Arvore unica: cuidado com superajuste -> controle a complexidade (alpha de poda)
6 arvore = DecisionTreeRegressor(ccp_alpha=0.0) # ccp_alpha e o nosso alpha
7
8 # Floresta aleatoria: B grande, m = max_features (~ p/3 em regressao)
9 rf = RandomForestRegressor(n_estimators=500, max_features=1/3,
10                             oob_score=True, random_state=0).fit(X_tr, y_tr)
11 print("risco OOB (R2):", rf.oob_score_)
12 print("importancias:", rf.feature_importances_)
13
14 # Boosting: lambda pequeno + B por early stopping
15 gb = GradientBoostingRegressor(learning_rate=0.01, n_estimators=2000,
16                                 max_depth=3, validation_fraction=0.2,
17                                 n_iter_no_change=20).fit(X_tr, y_tr)
```

O notebook Aula prática 06 compara árvore, floresta e *boosting* no `superconductivity.csv`, e mede o piso $\rho v(\mathbf{x})$ da variância.

Resumo

- **Árvore:** partição recursiva (1); cresce gananciosamente (2) e poda por custo-complexidade (3) (α por CV). Interpretável, mas alta variância.
- Agregar reduz variância se os preditores são não-viesados e pouco correlacionados (Prop. 2.1).
- **Bagging** (4): árvores profundas em amostras bootstrap; OOB “de graça”; importância por redução de RSS.
- **Florestas:** *descorrelacionam* sorteando $m \approx p/3$ variáveis por nó; robustas a B ; adaptam-se à esparsidade.
- **Boosting:** ajuste sequencial dos resíduos; λ pequeno, p raso, B por *early stopping* (cuidado: B grande superajusta). XGBoost é a implementação de referência.

Leitura recomendada. [AME] §4.8 (árvores), §4.9 (*bagging* e florestas), §4.10 (*boosting*); [ISLP] Capítulo 8 (*Tree-Based Methods*).